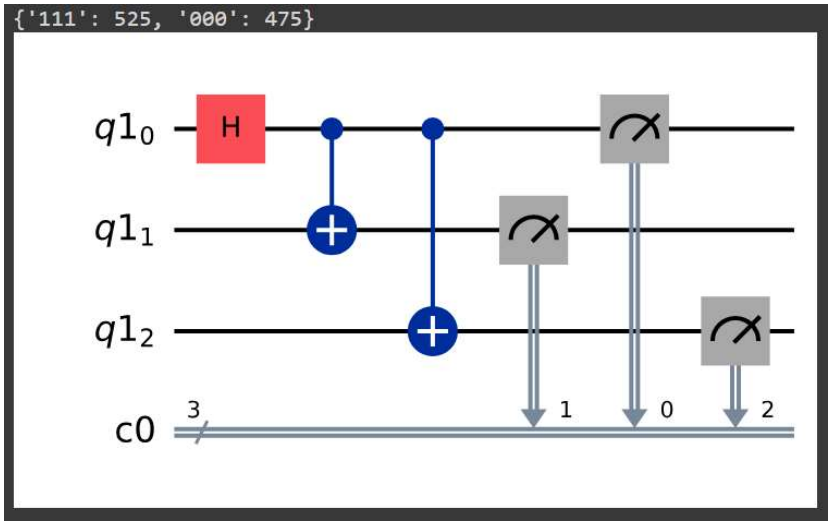
IQAP Project Report

Part 1 – QPE

1) Small Practice

```python
# Generating the circuit

q = QuantumRegister(3)   # We need 3 qubits..
c = ClassicalRegister(3) # ... and 3 bits to store the results
qc = QuantumCircuit(q,c) # the circuit !

# Bell state
qc.h(q[0])
qc.cx(q[0], q[1])
qc.cx(q[0], q[2])

qc.measure(q, c)

# Simulate the circuit
simulator = AerSimulator()
transpiled_qc = transpile(qc, simulator)
job = simulator.run(transpiled_qc, shots=1000)
result = job.result()
counts = result.get_counts()
print(counts)


qc.draw('mpl')
```

```
{'111': 525, '000': 475}
```

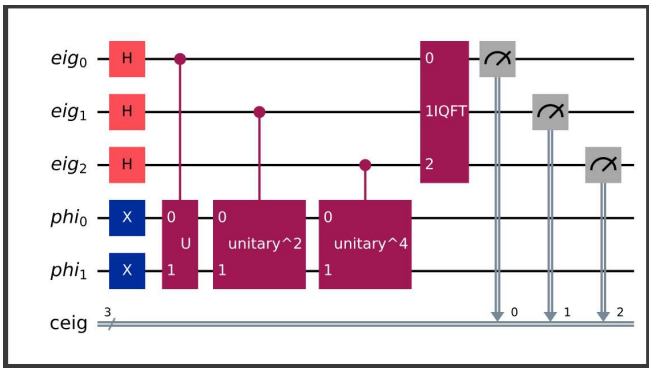


Repeated testing has shown consistency in the 50/50 split.

2.1) The given operator acts on two qubits, applying a phase shift to the |11> state that depends on the fraction used and leaving any other states untouched. The eigenvectors are |00>, |01>, |10>, and |11>, with eigenvalues of 1 for all but the last one which is exp(pi*2j*(fraction)). As for what should be returned with 3 bits of precision, it depends on the fraction : it should be 000 for 0, 001 for 1/8, 010 for 2/8… till 111 for 7/8, and it loops back to 000 at 8/8 and so on (ideally we want to stay under 1 though).

2.2)

```python
size_eig = 3
# size_eig = 4
# size_eig = 5 # number of qubits
size_phi = 2

eig = QuantumRegister(size_eig, name="eig")
phi = QuantumRegister(size_phi, name="phi")
ceig = ClassicalRegister(size_eig, name="ceig")
qc = QuantumCircuit(eig,phi,ceig)

# phi in |11> state
qc.x(phi[0])
qc.x(phi[1])

qc.h(eig)

# C-U, C-U^2, C-U^4, C-U^8, C-U^16
qc.append(U.control(), [eig[0]] + list(phi))
qc.append(U.power(2).control(), [eig[1]] + list(phi))
qc.append(U.power(4).control(), [eig[2]] + list(phi))
# qc.append(U.power(8).control(), [eig[3]] + list(phi))
# qc.append(U.power(16).control(), [eig[4]] + list(phi))

# Inverse QFT
qc.append(QFT(size_eig).inverse(), eig)

qc.measure(eig,ceig)
```



Output : {'011': 1024}

2.3) This is the expected result, as explained previously. And results are also coherent for the other values. This also holds true for 4 and 5 qubits with sixteenths and thirty-twoths. We used the unitary gate to encode the desired value, then extracted it through inverse QFT.

2.4) Applying this to 1/3 and trying to approximate gives interesting results : first of all, results is plural because we no longer have a singular identical one. They aren't equally distributed however : the most favored results are ones resembling a 0.0101010101... loop, only considering the right part. This is because it is equivalent to 1/3, as I'll briefly show here :

3 * 0.0101010101… = 2 * 0.0101010101… + 0.0101010101…
3 * 0.0101010101… = 0.1010101010… + 0.0101010101…
3 * 0.0101010101… = 0.1111111111…

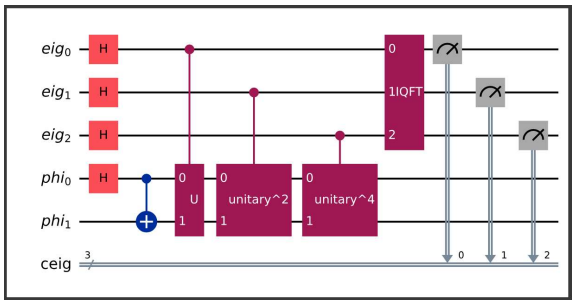
Using repeating decimal proof in a binary setting, we can then see :

3 * 0.0101010101… = 1
1/3 = 0.0101010101…

That being said, the approximation leads to some very strange behaviour, and sticking to powers of 2 is preferrable.

2.5)

```
U = UnitaryGate(
    Operator([[1,0,0,0],
              [0,1,0,0],
              [0,0,1,0],
              [0,0,0,np.exp(pi*2j*(3/8))]]), label="U")

size_eig = 3
size_phi = 2

eig = QuantumRegister(size_eig, name="eig")
phi = QuantumRegister(size_phi, name="phi")
ceig = ClassicalRegister(size_eig, name="ceig")
qc = QuantumCircuit(eig, phi, ceig)

# Bell state with superposition
qc.h(phi[0])
qc.cx(phi[0], phi[1])

qc.h(eig)

qc.append(U.control(), [eig[0]] + list(phi))
qc.append(U.power(2).control(), [eig[1]] + list(phi))
qc.append(U.power(4).control(), [eig[2]] + list(phi))

qc.append(QFT(size_eig).inverse(), eig)

qc.measure(eig, ceig)
```



We get a roughly even distibution of 000 and 011, corresponding to the trivial and non trivial eigenvectors, respectively.

Part 2 – Shor

2.1)

```python
# a, p, N et n are as above
# we compute x |--> a^p * x mod N, seen as an operator on C^{2^n}

def gateMult(a,p,N,n):
    nn = 2 ** n
    M = [[0 for x in range(nn)] for i in range(nn)]

    # FILL IN M WITH SOMETHING
    for i in range(nn):
      if i < N :
        j = ((a ** p) * i) % N
        M[j][i] = 1
      else :
        M[i][i] = 1

    U = Operator(M)
    return(UnitaryGate(U))
```

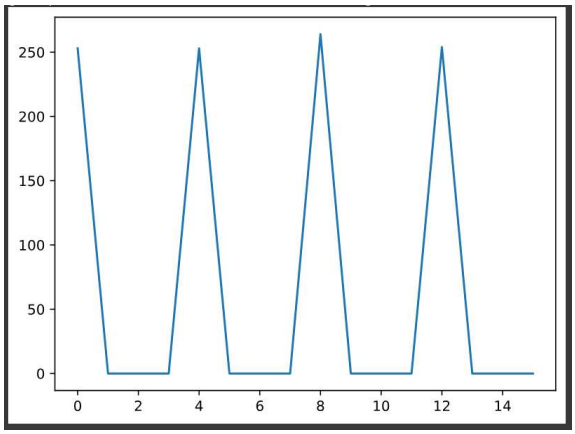
I have confirmed from testing the produced matrix aligns with what we want.

2.3)

3 – 137
4 – 564
5 – 2404
6 – 9867

There is roughly a 4 times increase per additional qubit, as such the complexity would be of the order $O(4^n)$. Since the circuit size increases exponentially, using this method for large values isn't a great idea. I do not know how we could implement this with larger values however.

3.2)



3.3) The order of 7 mod 30 is 4. s/r is the x value divided by size_eig. We can deduce r from the graph if we look at the largest common denominator of the x values of the peaks (exp : on the above graph, 4, 8 and 12 share 4 as ther largest common denominator). Values are very erratic for a = 20 and N = 29 when using this amount of qubits, as such a higher size_eig helps. Lastly, this seems to work for the values I've tested.